

Three New Engines for Forge

Constructive polyhedral projection,
noncommutative completion, and
zero-aware analytic certificates

A delta design with exact Python prototypes

$$z(x, y) = \max(0, x + y - 1),$$

$$p = \sum_j c_j u_j r_{ij} v_j,$$

$$f(x) = \frac{f^{(m)}(\xi)}{m!} (x - a)^m.$$

What is delivered. Three implemented research workers, separately implemented exact checkers, replayable certificates, adversarial controls, mathematical soundness arguments, and a concrete Lean integration plan.

What has not been measured. No Lean compiler was available in this execution environment. The Lean specimens are uncompiled, and no comparison with `grind` or another Lean tactic was run.

Abstract

Forge already has a substantial design for certificate-producing proof planning above Lean’s `grind`: induction and invariant synthesis, arithmetic witnesses, polynomial certificates, finite closures, and several external proof-search interfaces. Recommending these again would not significantly extend it. This study instead develops three comparatively independent additions.

The first turns exact polyhedral projection into a constructive procedure for universally quantified parameters and multiple existential real outputs. It accepts mixed strict and non-strict affine constraints and extracts a shared min–max expression rather than searching a fixed affine template. A four-corner example proves a strict expressiveness separation from affine witnesses. The second operates in a free associative algebra: a bounded, traced completion procedure discovers missing noncommutative lemmas, while a much smaller checker verifies only a two-sided ideal identity in the original relations. The third supplies rational enclosures for elementary functions and combines them with exact vanishing-jet certificates, so a non-strict inequality with an interior zero need not be defeated by numerical uncertainty at that zero.

All three workers were prototyped and exercised. Their separate workloads contain 110 polyhedral cases, 85 noncommutative cases, and 44 analytic cases. Expected negative and resource-limited outcomes are retained rather than counted as failures or silently discarded. Replay is performed with search imports blocked and site packages disabled. Additional evidence comprises an exact vertex-enumeration oracle, polynomial-action checks, witness evaluations, mutation rejection, and optional numerical sanity checks. None of this is reported as a Lean-kernel proof. The final sections specify the source reification obligations, checker theorems, library reuse, and release gates required to turn these prototypes into actual Forge capabilities.

Contents

1	The contribution relative to the repository	3
1.1	The baseline is already large	3
1.2	Three different kinds of increased capability	3
1.3	One common interface, without a new global architecture	4
2	Constructive polyhedral projection	4
2.1	The fragment and its semantics	4
2.2	A valid specification that defeats every affine template	5
2.3	One elimination step	5
2.4	The certificate must check both directions	6
2.5	From projection to a global witness	7
2.6	Proving the parameter-domain implications	7
2.7	A real counterexample, not merely failure to synthesize	8
2.8	Cost, improvements, and precise library choices	8
2.9	Extensions deliberately not included in the executable	9
3	Noncommutative completion with original-relation certificates	9
3.1	Why this is a separate algebraic worker	9
3.2	Words, polynomials, and interpretation	10
3.3	A certificate that does not require a completed basis	10
3.4	A minimal completion-only example	11
3.5	The traced completion algorithm	11
3.6	Termination claims that are and are not justified	12
3.7	Examples with mathematical structure	13
3.8	Concrete library integration	14
3.9	Search improvements that preserve the same checker	14
4	Elementary-function certificates that preserve exact zeros	14
4.1	Two obstacles, not one	15
4.2	The admitted grammar	15

4.3	Exact rational interval arithmetic	15
4.4	A small library of rational elementary bounds	16
4.5	Coverage certificates rather than sampled grids	17
4.6	Vanishing jets: remove the uncertain value at the zero	17
4.7	Four worked certificates	18
4.8	Automatic proposal without guessing exact zeros	19
4.9	The exact meaning of checker independence	19
4.10	A limited completeness theorem	20
4.11	A stronger production lane: polynomial Taylor models	20
4.12	Library choices and range reduction	21
5	Integration into Forge, Leant, and Djex	21
5.1	Keep the existing controller; add three typed adapters	21
5.2	The polyhedral source bridge	22
5.3	The noncommutative source bridge	22
5.4	The analytic source bridge	23
5.5	Concrete cooperation: a witness plus an analytic guarantee	23
5.6	A longer-term operator application with an explicit bound	23
5.7	How Leant and Djex benefit without changing their type theories	24
6	Executed evidence and its limits	24
6.1	Environment and reproducibility	24
6.2	Keep the units and outcomes separate	25
6.3	Polyhedral workload	25
6.4	Noncommutative workload	25
6.5	Analytic workload and automatic rediscovery	26
6.6	Mutation and structural controls	26
6.7	Replay with search unavailable	27
6.8	Timing and size data	27
6.9	What the evidence establishes	27
7	An implementation sequence with falsifiable release gates	28
7.1	Gate 1: promote exact polyhedral receipts to Lean theorems	28
7.2	Gate 2: noncommutative certificates before completion in Lean	28
7.3	Gate 3: analytic atoms, then exact jets	28
7.4	Gate 4: composition through the obligation broker	29
7.5	Evaluation that would justify a stronger claim	29
7.6	Recommended scope decision	29
A	Artifact guide and source provenance	30
A.1	What is in the archive	30
A.2	Running the supplied code	30
A.3	Pinned repositories and reviewed surfaces	31
A.4	Reproduction and interpretation notes	31
	References	32

1 The contribution relative to the repository

1.1 The baseline is already large

The reviewed Forge snapshot is a design with prototype workers, not an installed Lean tactic. Its consolidated article merges eighteen contributions in two rounds. The implementation ledger separates executed Python experiments, generated Lean source, and designs that have not been implemented. In particular, the absence of an integrated tactic is not the absence of useful algorithms: the repository already contains a considerable collection of them [1, 2].

This report takes those designs as prerequisites. It does not propose a new scheduler, another global “untrusted search, trusted checker” architecture, a renamed polynomial cone, or another generic call to an SMT solver. The audit covered the current README and status ledger, the provenance map, the consolidated witness and nonlinear sections, the Leant/Djex transfer document, and the runtime contract. Leant’s README and Djex’s rank-N account were also inspected at the revisions recorded in Appendix A. The detailed witness and nonlinear accounts and the existing transfer ledger were treated as prior work [3, 4, 5]. This is a targeted audit of the consolidated baseline, not a claim to have independently reviewed every line of all eighteen archived submissions.

Area	Already in the reviewed baseline	Added here
Real witnesses	Fixed affine templates with Farkas multipliers; a general allowance for proved piecewise branches	An exact projection-and-lifting algorithm for all outputs in a mixed affine conjunction, with an explicit min-max witness DAG and complete pair coverage
Integer witnesses	Lattice elimination, residue witnesses, and one-output integer projection with Bézout/CRT evidence	Not reimplemented. The new worker is over ordered fields, not integers; midpoint reconstruction is not an integer operation
Algebra	Commutative polynomial ideal certificates; specialized Ore-operator transport for recurrences	General free-associative word polynomials, traced overlap/inclusion completion, and two-sided original-relation certificates
Inequalities	Polynomial cones, quadratics, Bernstein subdivision, square factors, and Sturm reasoning	Elementary-function enclosures, exact analytic domain obligations, and vanishing-jet certificates for non-strict zeros
System integration	Scope-aware obligations, candidate admission, trust policies, and worker isolation	Three concrete adapters with new semantic contracts; the existing controller remains in place

The comparison is with the reviewed Forge design, not with the whole literature. Fourier–Motzkin elimination, noncommutative completion, and verified Taylor approximation are established techniques. Their value here is that they can be turned into small, auditable, composable Forge workers, with precise capability boundaries. The specific certificate contracts, separation examples, implementation choices, and experiments are this report’s contribution.

1.2 Three different kinds of increased capability

The polyhedral worker increases *witness expressiveness*: some valid linear specifications have no affine witness at all. Spending more time on the existing affine coefficient template cannot repair that mismatch.

The noncommutative worker increases *lemma discovery*: a supplied orientation of the hypotheses can be terminating yet incomplete for the target. Critical pairs expose consequences that no direct reduction of that target initially uses.

The analytic worker increases *semantic coverage*: a polynomial engine cannot infer the defining properties of an opaque $\exp(x)$ atom without additional theorems. Furthermore, even valid rational enclosures can be insufficient at a zero of the desired inequality. Exact vanishing information addresses that obstruction rather than merely increasing numerical precision.

These distinctions determine the tests. We use an impossibility proof for affine witnesses, a completion-on/off experiment on identical relations, and interval-versus-jet experiments on identical elementary expressions. These are mechanism-level comparisons. They are not claims that stock grind, Mathlib’s `noncomm_ring`, or another existing tactic fails on the examples. The official grind manual and Mathlib tactic documentation remain the right starting points for an actual Lean baseline [9, 13].

1.3 One common interface, without a new global architecture

Each worker consumes a sealed, typed problem and returns an untrusted object. The existing Forge admission boundary must still relate that object to the original Lean expression. What differs is the proposition its checker should establish:

$$\begin{aligned} \text{Polyhedral: } & D(x) \implies R(x, W(x)); \\ \text{Noncommutative: } & r_1 = \dots = r_s = 0 \implies p = 0; \\ \text{Analytic: } & x \in B \implies f(x) \geq 0. \end{aligned}$$

For the polyhedral worker there is also a genuine negative object: a rational parameter value satisfying the domain but admitting no existential output. The other two prototypes do not emit semantic refutations. A nonzero noncommutative remainder or an inconclusive interval is only UNKNOWN.

A useful implementation order. First add constructive polyhedral projection, because its completeness statement is clean and its Lean proof obligations are elementary. Next add noncommutative certificates, whose positive checker is particularly small. Add the analytic worker after the rational arithmetic and source-domain infrastructure have been stabilized; its search is simple, but its proof bridge is substantially richer.

2 Constructive polyhedral projection

Evidence status. Implemented in Python, including multiple outputs, strict rows, exact counterexamples, separately implemented certificate checking, and witness-DAG extraction. Lean reification and checker soundness have not been implemented.

2.1 The fragment and its semantics

Let $x \in \mathbb{R}^p$ be parameters and $y \in \mathbb{R}^q$ existential outputs. The input is

$$\forall x, \quad D(x) \implies \exists y, R(x, y), \tag{1}$$

where D and R are finite conjunctions of rational affine inequalities, each either strict or non-strict. Equalities can be encoded by two non-strict inequalities. A row is a pair (a, σ) , with a constant coordinate appended to a , and means

$$a_0 z_0 + \dots + a_{n-1} z_{n-1} + a_n \begin{cases} \geq 0, & \sigma = 0, \\ > 0, & \sigma = 1. \end{cases}$$

The parameter domain must not mention output coordinates. There are no disequalities, disjunctions, products of variables, floor operations, or integer coercions in this first fragment.

The result is not a collection of sampled solutions. It is an explicit term $W(x)$, constructed from rational affine operations, min, and max, together with a finite proof that it meets the relation for every parameter in the domain. This is a constructive use of elimination: remove outputs while recording how to restore them. Mixed strict/non-strict Fourier–Motzkin elimination supplies the underlying projection operation [10].

2.2 A valid specification that defeats every affine template

Consider

$$0 \leq x, y \leq 1 \implies \exists z, \quad 0 \leq z, \quad x + y - 1 \leq z, \quad z \leq x, \quad z \leq y. \quad (2)$$

The four corners determine the value of any admissible witness there:

$$z(0, 0) = 0, \quad z(1, 0) = 0, \quad z(0, 1) = 0, \quad z(1, 1) = 1.$$

Proposition 2.1 (Affine obstruction). *No affine function $z = ax + by + c$ satisfies (2) on the whole unit square. Nevertheless, the continuous piecewise-affine function*

$$W(x, y) = \max(0, x + y - 1)$$

satisfies it everywhere on that square.

Proof. The first three corner values force $c = 0$, $a = 0$, and $b = 0$; the last then gives the contradiction $0 = 1$. For the proposed witness, both lower bounds follow from the definition of maximum. The two arguments of the maximum are each at most x : $0 \leq x$, and $x + y - 1 \leq x$ because $y \leq 1$. Hence $W \leq x$. The proof of $W \leq y$ is symmetric. $\square$

This is not a heuristic failure caused by a poor LP solver. The affine search space contains no solution. The new worker changes the representation of witnesses while retaining exact linear arithmetic for its proof obligations. The example also exposes a useful practical feature: the witness is continuous, so there is no need to invent a discontinuous branch split at $x + y = 1$.

2.3 One elimination step

Fix an output variable t , and write the other variables as z . Partition the rows by the sign of the coefficient of t . After dividing only by positive rationals, the relevant rows have the forms

$$t \geq_i L_i(z), \quad U_j(z) \geq_j t,$$

where $\geq$ is either $\geq$ or $>$. Keep every row independent of t . For each lower–upper pair create

$$U_j(z) - L_i(z) \begin{cases} \geq 0, & \text{both source rows are non-strict,} \\ > 0, & \text{at least one source row is strict.} \end{cases} \quad (3)$$

The strictness bit is a logical part of the row, not a formatting choice.

In the original coefficient representation, if $a_{it} > 0$ and $a_{jt} < 0$, the projected affine expression is

$$\frac{r_i}{a_{it}} - \frac{r_j}{a_{jt}}. \quad (4)$$

Both multipliers are positive. The prototype rescales a nonzero output row by a positive factor so that its first nonzero coefficient has absolute value one. It deduplicates equal normalized rows, retaining the strictness bit. Rescaling by a negative factor would reverse the inequality and is forbidden.

Theorem 2.2 (Exact one-variable projection). *For every assignment to z , the projected conjunction holds if and only if there exists $t \in \mathbb{R}$ satisfying the original conjunction.*

Proof. The forward implication from an original solution to every projected row follows by the positive combination (4). If one contributing inequality is strict, its positive contribution makes the sum strict. Independent rows are unchanged.

For lifting, suppose the projected conjunction holds. If both lower and upper bounds are present, set

$$L = \max_i L_i(z), \quad U = \min_j U_j(z).$$

All pairs imply $L \leq U$. If every bound on t is non-strict, choose $t = L$. If any is strict, choose $t = (L + U)/2$. When $L < U$, the midpoint lies strictly between the extrema and meets every strict bound. When $L = U$, every strict lower bound is strictly below U , and every strict upper bound is strictly above L , by its pair rows. Thus the same midpoint still satisfies all strict bounds. This last case matters: a nonactive strict bound need not force the two active extrema apart.

With only lower bounds, choose their maximum if all are non-strict, and that maximum plus one otherwise. With only upper bounds, use their minimum or that minimum minus one. With no bounds on t , use zero. In every case, the independent rows remain true. $\square$

For example, $t > 0, t \geq 1, t \leq 1$ is feasible with $t = 1$. An implementation that sees “some strict bound” and therefore demands $L < U$ would incorrectly reject it. This exact touching-extrema case is included in the experiments.

2.4 The certificate must check both directions

An elimination trace that checks only nonnegative combinations establishes

$$R(z, t) \implies Q(z).$$

That is sufficient for some refutations but insufficient for witness extraction. A malicious or incomplete producer could delete every projected row, present $Q = \text{True}$, and thereby make every parameter look feasible. The lifting implication must be checked separately.

Each step therefore contains two complementary components. Every published output row comes with exact nonnegative weights expressing it as a combination of the input rows. In addition, a coverage table names *every* zero-coefficient row and *every* lower–upper pair, and points to an output row that is a positive scalar multiple of the required row with exactly the required strictness.

The independent checker reconstructs the set of required indices from the input coefficients, rejects missing or repeated coverage entries, and tests the pointed-to rows arithmetically. It does not call the projection assembler. Multiple required pairs may share one normalized output row. Additional output rows are harmless only because their nonnegative-combination receipts separately prove they are consequences of the input.

This format makes the two directions operationally visible. The implication direction is a linear identity; the lifting direction is a complete local enumeration. The latter is the essential addition beyond a usual Farkas-style proof of an already selected affine witness.

```

project_checked(rows, variable):
  P, N, Z := partition rows by coefficient sign
  required := all Z rows and all pairs in P x N
  outputs := normalized required rows, with exact deduplication
  for each output:
    record nonnegative input-row multipliers
  for each required item:
    record its output index
  return outputs and the two-direction receipt

```

2.5 From projection to a global witness

Eliminate the output variables in a fixed order. At the end, the projected relation $Q(x)$ contains only parameters and is exactly equivalent to $\exists y, R(x, y)$. Prove each row of Q from the parameter domain. Then restore the eliminated variables in reverse order using Theorem 2.2.

The reverse order is not an implementation detail. When eliminating y_q , its bounds may mention $y_1, \dots, y_{q-1}$. Those variables must already have been reconstructed when y_q is restored. The emitted expression graph enforces this dependency order: a node may refer only to parameters and earlier available nodes.

The witness graph has six constructors:

$$\text{Const}(c), \quad \text{Param}(i), \quad \text{Add}(u, v), \quad \text{Scale}(c, u), \quad \text{Max}(u, v), \quad \text{Min}(u, v).$$

It is hash-consed and emitted in topological order. A midpoint is an addition followed by scaling by $1/2$; one-sided slack is addition of 1 or -1 . There is no infinity literal and no division by a variable. The prototype's `compile_witness` first invokes the independent positive checker and then deterministically rebuilds this graph. It never accepts an arbitrary source program supplied alongside the certificate.

Corollary 2.3 (Piecewise-affine selection). *Every satisfiable instance of (1) has a witness expressible by the finite graph above. The extracted witness is continuous and piecewise affine as a function of the parameters.*

Proof. Every normalized bound is affine in the currently remaining variables. Reverse reconstruction substitutes previously obtained continuous piecewise-affine expressions into such bounds and combines them with addition, rational scaling, finite minimum, and finite maximum. These operations preserve continuity and piecewise affinity. Induction over the reverse elimination schedule proves the statement. The witness is guaranteed to satisfy the relation only on D , although the expression itself is defined on all real parameter values. $\square$

For the coupled test family, an additional output obeys $w = 2z + x$. Both outputs are eliminated and reconstructed. The procedure does not first guess z , freeze that guess, and hope to solve for w ; it projects the simultaneous relation and restores a compatible tuple.

2.6 Proving the parameter-domain implications

For a projected row r , test the feasibility of $D \wedge \neg r$. Negation must switch strictness:

$$\neg(a \geq 0) \equiv -a > 0, \quad \neg(a > 0) \equiv -a \geq 0.$$

The prototype uses the same mathematical elimination procedure as a search engine, carrying nonnegative origin weights through each step. Its *checker*, however, verifies only the final combination of the original domain rows and the negated target row.

A contradiction receipt consists of weights $\lambda_i \geq 0$ such that the variable coefficients vanish and the resulting constant is negative, or is zero with at least one strictly positive weight on a strict input row. In formulas, if

$$\sum_i \lambda_i r_i(z) = c,$$

then a contradiction is obtained when $c < 0$, or when $c = 0$ and $\exists i, \lambda_i > 0 \wedge \sigma_i = 1$. A strict input with weight zero must not make the sum strict. The test suite contains precisely this control.

This handles empty parameter domains automatically: an inconsistent D proves every projected row without requiring an arbitrary witness to satisfy an impossible premise. Empty-domain cases are recorded separately from nonempty-domain successes in the generator, rather than used to inflate an apparent capability result.

2.7 A real counterexample, not merely failure to synthesize

If $D \wedge \neg r$ is feasible, the feasibility search reconstructs a rational point x_0 . The negative certificate contains that parameter assignment, the projection trace, and the index of the violated projected row. The checker verifies

$$D(x_0), \quad \neg r(x_0), \quad R(x, y) \implies Q(x).$$

Consequently no output tuple can satisfy $R(x_0, y)$. Unlike a failed affine-template LP, this is a counterexample to the universally quantified specification itself.

Theorem 2.4 (Unbounded-fragment completeness). *With exact rational arithmetic and no resource caps, the projection-and-lifting algorithm terminates on every input in the stated fragment. It either constructs a witness for (1) or constructs a rational parameter counterexample.*

Proof. Each elimination removes one variable and produces a finite collection of rows. Theorem 2.2 preserves feasibility in both directions. Eliminating every variable therefore reduces feasibility to finitely many constant comparisons, and reverse reconstruction produces a rational solution whenever those comparisons are consistent. Apply this procedure to the finite collection of domain implication tests. If all are inconsistent, their contradiction receipts prove $D \implies Q$, and reverse lifting supplies a witness. Otherwise the rational solution of one implication test gives the stated counterexample. $\square$

The concrete executable imposes a projection pair budget and finite decoder bounds. Those versions are intentionally incomplete and return UNKNOWN when a search cap is reached. The theorem describes the underlying algorithm, not an assurance that a fixed-budget call always decides its input.

2.8 Cost, improvements, and precise library choices

If a step has m_+ positive, m_- negative, and m_0 zero rows, it creates at most

$$m_+ m_- + m_0$$

required items before deduplication. Repeated projection can produce doubly exponential growth. The prototype chooses a fixed reverse coordinate order for reproducibility, not because that order is optimal.

A production implementation should choose a pivot minimizing the pair count, with a tie-breaker based on support size and coefficient bit lengths. The checker then verifies the explicit permutation and dependency schedule. Exact duplicate elimination is already implemented. Redundancy deletion is a separate extension: deleting a nonduplicate row requires an implication certificate

from retained rows, including the correct strictness condition. Omitting a required lower–upper pair because a numerical LP regards it as redundant is not acceptable evidence.

Dense origin-weight vectors are convenient but can dominate memory. Replace them with a sparse addition/scaling DAG and replay it in topological order. The final proof need not expand every derivation into an exponentially duplicated Lean term. This is a lane-specific reason to use shared proof terms, not a proposal to replace Forge’s existing proof-cost policy.

For the initial port, Lean’s rational and integer arithmetic plus small proved row-combination lemmas are sufficient. `linarith` can be used as a reconstruction aid for small source-level specimens, but the final reflected checker should evaluate the supplied exact multipliers rather than rerun an unconstrained proof search. External exact LP or polyhedral tools are optional proposal engines, not dependencies of the supplied prototype. Fourier–Motzkin was chosen here because it makes witness reconstruction and strictness auditing transparent; it is not a claim that it is the fastest feasibility algorithm.

2.9 Extensions deliberately not included in the executable

Boolean specifications can be handled by finite branch decomposition followed by guarded choice, but a branch’s projected domain must be proved, and the union of branch domains must cover D . A decision tree of witnesses is not licensed merely because each leaf works somewhere. Reusing the new polyhedral projection objects as exact guards would provide the right ingredients; the Boolean controller is not implemented here.

Alternating quantifiers require a further distinction. A witness for $\exists u \forall v \exists w$ cannot refer to v , while the witness for w may. The current single parameter block and single output block avoid that dependency problem. A future quantified-real worker must preserve the original quantifier order and use strategy-like witnesses, rather than treating every displayed variable as a parameter.

Integer outputs must use the existing lattice/residue machinery or a separate Presburger synthesis procedure. Replacing a real midpoint by truncating division is unsound. These exclusions keep the completeness statement narrow enough to be proved and broad enough to add a genuine capability.

3 Noncommutative completion with original-relation certificates

Evidence status. Implemented in Python: sparse rational word polynomials, overlap and inclusion compositions, traced reduction, bounded completion, and an independent direct identity checker. No Lean reifier or reflected noncommutative checker was compiled.

3.1 Why this is a separate algebraic worker

The commutative identity $xy = yx$ is not a harmless normalization rule for matrices, endomorphisms, differential operators, or elements of a presented associative algebra. A commutative polynomial representation erases precisely the information needed to reason about these objects.

Forge’s existing Ore-operator transport is related but different. It is specialized to recurrence operators and their coefficient-shift rules. The worker here accepts arbitrary finite sets of rational polynomial relations in a free associative algebra and seeks consequences in their two-sided ideal. It does not replace the specialized Ore algorithms when their additional structure is useful.

Mathlib already has `noncomm_ring`; it handles noncommutative ring expressions, and its documented syntax permits hypothesis-directed simplification. Thus “add noncommutative normalization” would not be an adequate proposal. The additional mechanism is *completion-driven generation of missing consequences*, followed by a certificate in the original hypotheses. Existing normalization is a building block for replay, not an opponent the report claims to beat [13].

3.2 Words, polynomials, and interpretation

Fix a finite alphabet $\Sigma = \{0, \dots, d-1\}$. A word is a finite list of letters, including the empty word ϵ . A polynomial is a finitely supported map

$$p : \Sigma^* \longrightarrow \mathbb{Q}.$$

Addition combines coefficients of identical words. Multiplication concatenates words:

$$(pq)(w) = \sum_{uv=w} p(u)q(v).$$

The empty word is the multiplicative unit. Word order is preserved throughout; the keys $(0, 1)$ and $(1, 0)$ are different.

Let A be a unital associative $\mathbb{Q}$ -algebra, and assign an element $\rho(i) \in A$ to each generator. Define

$$\llbracket a_1 \cdots a_k \rrbracket_\rho = \rho(a_1) \cdots \rho(a_k), \quad \llbracket p \rrbracket_\rho = \sum_w p(w) \cdot \llbracket w \rrbracket_\rho.$$

Rational coefficients act centrally through the algebra structure. This is the semantic reason for the $\mathbb{Q}$ -algebra restriction. Interpreting a rational coefficient in an arbitrary ring by an unguarded division operation would not establish the same theorem.

The input consists of original relations r_i interpreted as zero and a target difference p . A goal $s = t$ is represented by $p = s - t$. The source reifier must prove that this interpretation matches the actual Lean multiplication, addition, scalar action, and atoms in scope.

3.3 A certificate that does not require a completed basis

The positive certificate is a finite list of tuples

$$(c_j, u_j, i_j, v_j), \quad c_j \in \mathbb{Q}, \quad u_j, v_j \in \Sigma^*,$$

satisfying the formal identity

$$p = \sum_j c_j u_j r_{i_j} v_j. \tag{5}$$

Unlike the polyhedral multipliers, the coefficients here have no sign restriction.

Theorem 3.1 (Two-sided certificate soundness). *If (5) holds as an exact word-polynomial identity and every original relation evaluates to zero, then $\llbracket p \rrbracket_\rho = 0$ in every unital associative $\mathbb{Q}$ -algebra.*

Proof. Evaluation preserves rational linear combinations and word concatenation. Evaluating the right-hand side gives

$$\sum_j c_j \cdot (\llbracket u_j \rrbracket_\rho \llbracket r_{i_j} \rrbracket_\rho \llbracket v_j \rrbracket_\rho) = 0.$$

The formal identity identifies this value with $\llbracket p \rrbracket_\rho$. □

This theorem deliberately does not mention a monomial order, critical pairs, termination, or confluence. Those concepts help the producer find the certificate. Once the identity is supplied, none is needed to check it.

The checker in `prototype/forge_delta/nc_check.py` parses the externally supplied target and original relations, multiplies each original relation by the recorded left and right words, accumulates exact coefficients, removes zero coefficients, and compares the resulting dictionary with the target. It does not import the reducer or completion engine. Reversing a left context into a right context is rejected when it changes the identity.

3.4 A minimal completion-only example

Assume

$$ab = a, \quad ba = b.$$

Orient these as reductions $ab \rightarrow a$, $ba \rightarrow b$, and ask for $a^2 = a$. The target difference $a^2 - a$ contains neither left-hand side, so direct reduction does nothing. However, the overlap word aba reduces in two ways:

$$(ab)a \rightarrow a^2, \quad a(ba) \rightarrow ab \rightarrow a.$$

Their disagreement produces the missing rule $a^2 \rightarrow a$.

Writing $r_1 = ab - a$ and $r_2 = ba - b$, the final certificate is

$$a^2 - a = r_1 - r_1a + ar_2. \quad (6)$$

Expanding the right-hand side cancels the two occurrences of aba and the two of ab , leaving the desired target. This identity is what the checker sees; it need not reconstruct how the ambiguity was discovered.

The executable ablation is exact: with completion disabled, the result is UNKNOWN with two initial rules and no reduction steps. With completion enabled, a third rule is found and the target is certified. On the recorded run the search counted three examined compositions and two reductions. These tiny counts are implementation diagnostics, not performance estimates for an arbitrary algebra problem.

3.5 The traced completion algorithm

The prototype uses degree-lexicographic order: first compare word lengths, then compare the integer letters lexicographically. For a nonzero polynomial, let $\text{LM}(p)$ be its greatest word. Divide by its leading rational coefficient to obtain a monic rule

$$\ell - f = 0,$$

where every word in f is strictly below ℓ . A rule carries both its polynomial and a representation of that polynomial in the original relations.

Suppose a polynomial contains a term $c u \ell v$. Subtract

$$c u(\ell - f)v$$

and add the same contextual multiple of the rule's original-relation receipt to the reduction trace. If the initial polynomial is p_0 and the current residual is p , the invariant is

$$p_0 - p = \sum_j c_j u_j r_{i_j} v_j. \quad (7)$$

Every elementary reduction is therefore already proof-producing.

For two monic rules f and g , the engine considers two kinds of ambiguity.

Overlap. If $\text{LM}(f) = us$ and $\text{LM}(g) = sv$ for a nonempty shared word s , form

$$fv - ug.$$

The greatest overlap word cancels. Reduce the result by the current rules.

Inclusion. If $\text{LM}(f) = u \text{LM}(g)v$, form

$$f - ugv.$$

This also cancels a leading occurrence. Inclusion must be considered separately; suffix-prefix overlaps alone do not cover a leading word occurring strictly inside another.

Each composition has an original-relation receipt obtained by contextual addition and subtraction of the two parent receipts. If its reduced residual is nonzero, subtract the reduction trace from that receipt and normalize both by the residual's leading coefficient. This creates a new monic rule with a checked algebraic meaning. Re-enqueue its pairs with the existing rules and retry the target.

```
complete_for_target(relations, target, budgets):
  rules := monic original relations, with unit provenance
  try target reduction
  queue := all ordered pairs of rules
  while queue is not empty:
    enforce budgets
    take a pair
    for each overlap or inclusion composition:
      residual, reduction_trace := reduce(composition)
      if residual is nonzero:
        provenance := composition_trace - reduction_trace
        add normalized residual with normalized provenance
        enqueue pairs involving the new rule
        retry target reduction
      if target residual is zero:
        return original-relation identity
  return UNKNOWN
```

The algorithm is a bounded, target-oriented instance of the noncommutative completion tradition associated with Gröbner–Shirshov bases and the Diamond Lemma. The theoretical framework is established; what matters for Forge is that the positive output is reduced to (5) rather than requiring the kernel to trust the implementation of completion [11, 12].

3.6 Termination claims that are and are not justified

A fixed finite set of the oriented rules terminates under this word order. Replacing an occurrence of a leading word by strictly smaller words decreases the corresponding well-founded multiset measure on the polynomial support. This is compatible with left and right contexts because degree-lexicographic order is a monomial order on words.

The *completion process* need not terminate. A finitely presented noncommutative algebra need not have a finite completed basis in the chosen generators and order. The commutative polynomial ring's Noetherian termination argument cannot simply be reused. The prototype therefore caps reduction steps, accumulated term/provenance size, rules, and examined compositions.

If a finite converged basis has been established and its critical-pair conditions verified, a normal-form procedure can decide equality in the formal presented quotient. That is stronger evidence than this prototype exports. In particular, a nonzero residual under an incomplete set of rules does not prove even formal nonmembership.

There is another semantic restriction even after a complete quotient analysis. A nonzero element of the *universal presented algebra* can still map to zero under the particular interpretation in the user's Lean goal. For example, with no relations the word polynomial $ab - ba$ is nonzero, but a particular pair of matrices may commute. A negative result about the presentation is not

automatically a proof of $ab \neq ba$ for those matrices. The shipped worker returns `UNKNOWN` for both situations rather than crossing that boundary.

3.7 Examples with mathematical structure

3.7.1 Weyl normal ordering

Let D and X satisfy

$$DX - XD = 1.$$

For positive integers m, n , the tested target is

$$D^m X^n = \sum_{k=0}^{\min(m,n)} \binom{m}{k} \frac{n!}{(n-k)!} X^{n-k} D^{m-k}. \quad (8)$$

For each concrete pair $1 \leq m, n \leq 6$, the generator forms the two sides and the engine returns an original-relation identity. The checker verifies that identity exactly. The test generator's closed formula is not used by the checker.

There is also a genuinely different computational oracle: interpret X as multiplication by the polynomial variable and D as differentiation, then act on $1, x, \dots, x^7$. Operators are applied rightmost first, and polynomial degree is allowed to grow as needed. The resulting 288 action checks agree with the certificates.

It would be a mistake to substitute fixed finite-dimensional matrices with a truncated derivative/multiplication action and assume they satisfy the Weyl relation. In characteristic zero, $\text{tr}(DX - XD) = 0$, whereas $\text{tr}(I) = d$ for a nonzero finite dimension d . Boundary truncation necessarily breaks the relation. The oracle avoids that issue by using exact actions on individual monomials without a truncation boundary. These finite action tests are still sanity checks, not a proof of the general normal-ordering formula; the formal identity receipts establish the individual tested instances.

3.7.2 The quantum plane

For a fixed rational q , use the relation

$$YX = qXY.$$

A word with r occurrences of X , s occurrences of Y , and l inverted Y -before- X pairs should normalize to

$$q^l X^r Y^s.$$

The workload contains forty deterministic pseudo-random choices of a rational q and a word of length between three and twelve. Some words are already normal and yield an empty identity certificate; that is legitimate, not missing evidence. Changing the target by adding the constant one makes the saved certificate fail.

A symbolic q is not silently covered by these rational instances. A production extension can use a central polynomial coefficient ring with monic relations, or add explicit commutation relations for a distinguished parameter. If leading coefficients must be inverted, the corresponding nonvanishing or invertibility obligations must be carried into Lean.

3.7.3 Clifford-style relations

Take generators satisfying $e_i^2 = 1$ and $e_i e_j + e_j e_i = 0$ for distinct indices. The tested family proves

$$(e_1 + \dots + e_d)^2 = d, \quad 2 \leq d \leq 6.$$

This demonstrates cooperation among many original relations and cancellation of cross terms. It does not formalize a Clifford algebra construction or prove its universal properties. It proves the displayed consequence in every interpretation satisfying the supplied relations.

3.8 Concrete library integration

GAP's GBNP package is an especially well-matched optional producer. Its documented `SGrobnerTrace` computes traceable noncommutative bases, `StrongNormalFormTraceDiff` supplies a trace for a reduction difference, and `EvalTrace` evaluates a trace in the original relations. The trace tuples already distinguish left context, relation index, right context, and coefficient [12].

An adapter should decode those structured traces directly, convert their generator numbering and coefficient representation, and pass the result to Forge's independent checker. It should not scrape `PrintTraceList` output, and it should not use GBNP's own `EvalTrace` as its only checker. The supplied prototype has no GAP dependency; GBNP is a concrete larger-scale search replacement, not something executed in this study.

For Lean, `FreeAlgebra` provides the semantic object, and `Mathlib` exposes a free-monoid basis suitable for connecting a sparse word representation to algebraic semantics [14]. A practical first replay path expands a small certificate into contextual hypothesis equalities and discharges the remaining ring rearrangement with `noncomm_ring`. The scalable path proves one interpretation theorem for a sparse word-polynomial checker, reducing each later replay to exact coefficient arithmetic plus Theorem 3.1.

The source reifier should initially accept only a single multiplication carrier with a known associative algebra structure. Function application is not automatically multiplication in an endomorphism algebra. Rectangular matrix products need a typed-path or multi-sorted extension. Inverses, adjoints, and transposes require explicit interpretation rules or supplied relations. Treating them as opaque generator expressions is safe for positive consequences, but loses the identities that would require analyzing their meaning.

3.9 Search improvements that preserve the same checker

The reference engine scans words and rule left-hand sides naively. A production implementation can index leading words by a prefix trie and maintain occurrence information, reducing repeated substring scans. Critical pairs can be prioritized by the degree of their overlap and by whether their residual words occur in the current target support. A deterministic age quota is needed if the intended search claim depends on eventual processing of deferred pairs.

Certificate size may be a worse bottleneck than polynomial size. Store rule provenance as a DAG of contextual linear combinations rather than eagerly expanding it after every reduction. Before export, retain only nodes reachable from the target proof. The final checker can replay the DAG with memoized sparse polynomials or accept a flattened certificate when it is already small. All of these changes preserve the theorem checked by the current direct evaluator.

The important engineering separation is therefore unusually sharp: the search can grow from this small Python implementation to GBNP or a specialized normal-ordering engine without changing the meaning of an accepted positive certificate.

4 Elementary-function certificates that preserve exact zeros

Evidence status. Implemented in Python for one real variable: rational interval evaluation of a fixed elementary-expression grammar, checked coverage trees, exact jets, and bounded automatic anchor/order discovery. General Taylor-model arithmetic, multivariate certificates, and Lean soundness proofs are proposed extensions.

4.1 Two obstacles, not one

A polynomial certificate engine can reason about $u = \exp(x)$ as a fresh variable, but it cannot obtain useful relationships between u and x merely by expanding polynomials. A semantic bridge must supply proved elementary-function facts.

A second obstacle remains even after adding rigorous numerical bounds. Suppose the goal is

$$f(x) \geq 0 \quad (x \in B),$$

and f actually vanishes at an interior point. An enclosure of the form $[f(x) - \varepsilon, f(x) + \varepsilon]$, however small its positive error, does not prove nonnegativity at that zero. Subdividing around the point may leave a leaf containing it forever. The appropriate remedy is to expose an exact vanishing factor or derivative structure, not to interpret a sufficiently small negative lower bound as zero.

Verified Taylor/interval methods are established in other proof assistants. Solovyev and Hales describe formal verification of nonlinear inequalities with Taylor interval approximations, while the Rocq Interval package supplies an existing proof-producing interval methodology [15, 16]. The proposal here is a Forge adapter with a deliberately small initial rational grammar and a zero-aware certificate constructor. It is not a claim to invent validated numerics.

4.2 The admitted grammar

The executable accepts expression trees generated by

$$e ::= c \mid x \mid -e \mid e + e \mid ee \mid e/e \mid e^n \mid \exp(e) \mid \log(e) \mid \sin(e) \mid \cos(e),$$

where $c \in \mathbb{Q}$ and $n \in \mathbb{N}$ is a literal. The problem is an original expression and a closed rational interval $B = [\ell, u]$, asking for $e(x) \geq 0$ throughout B .

Every division requires an interval denominator excluding zero. Every logarithm requires a strictly positive interval argument. These are admission conditions for the analytic lemmas, not assertions that unsupported source expressions are false. In particular, Lean's use of total functions does not remove the side conditions of the real-analytic theorems used to justify differentiation and series bounds.

The checker traverses the *original* expression tree on every leaf, even for a jet proof whose derivative has simplified dramatically. This prevents a simplifier from erasing the domain of a subexpression. The explicit expression $0/x$ on $[-1, 1]$ is rejected by this analytic fragment: rewriting a displayed numerator to zero does not establish the nonvanishing premise of the reciprocal derivative rules.

The implementation is univariate. Expressions may be nested, but there is only one free real coordinate. A multivariate natural-interval extension is straightforward at the representation level; the higher-order derivative and vanishing certificates require additional care and are not claimed to exist in the supplied code.

4.3 Exact rational interval arithmetic

For intervals $A = [a_-, a_+]$ and $B = [b_-, b_+]$, the checker implements

$$\begin{aligned} A + B &= [a_- + b_-, a_+ + b_+], \\ -A &= [-a_+, -a_-], \\ AB &= [\min S, \max S], \quad S = \{a_-b_-, a_-b_+, a_+b_-, a_+b_+\}. \end{aligned}$$

If $0 \notin B$, then

$$B^{-1} = [1/b_+, 1/b_-].$$

Integer powers use endpoint monotonicity; an even positive power has lower bound zero when its argument interval crosses zero. The zeroth power is exactly $[1, 1]$.

All endpoints are rational numbers represented by arbitrary-precision integer numerator and positive denominator. There is no floating-point rounding mode to configure in the checker. This makes exact replay simple, though potentially expensive when high-order expansions create large denominators. Bit-length and operation budgets remain necessary in a production implementation.

4.4 A small library of rational elementary bounds

The first implementation uses expansions around zero, except for a transformed logarithm. They are intentionally elementary enough to prove independently and to recompute without calling a transcendental numerical library.

4.4.1 Exponential

For a rational point a , let

$$T_N(a) = \sum_{k=0}^N \frac{a^k}{k!}, \quad r = |a|.$$

If $r < N + 2$, define

$$E_N(r) = \frac{r^{N+1}}{(N+1)!} \frac{1}{1 - r/(N+2)}. \quad (9)$$

Then

$$T_N(a) - E_N(r) \leq \exp(a) \leq T_N(a) + E_N(r). \quad (10)$$

Proof. The absolute value of the omitted term of index $N + 1$ is $r^{N+1}/(N+1)!$. Every subsequent ratio is at most $r/(N+2) < 1$. The absolute tail is therefore bounded by the geometric series in (9). Applying this to the exponential power series proves (10). $\square$

For an interval argument $[a, b]$, monotonicity gives a lower bound from the point enclosure at a and an upper bound from the point enclosure at b . The producer computes the Taylor polynomial by powers and factorials. The checker recomputes it using a term recurrence, retaining the same mathematically justified tail bound.

4.4.2 Logarithm

For $a > 0$, set

$$t = \frac{a-1}{a+1}, \quad |t| < 1.$$

Use

$$\log a = 2 \sum_{k=0}^{M-1} \frac{t^{2k+1}}{2k+1} + R_M, \quad |R_M| \leq \frac{2|t|^{2M+1}}{(2M+1)(1-t^2)}. \quad (11)$$

Proof. Integrate the geometric expansion of $1/(1-s^2)$, or equivalently use the odd power series for $2 \operatorname{artanh}(t)$. In the absolute tail, each denominator is at least $2M+1$; bounding it by that denominator leaves a geometric sum in $|t|^2$. The identity $a = (1+t)/(1-t)$ gives the logarithm formula. $\square$

Monotonicity again reduces an interval argument to two rational endpoint computations. No rational approximation to a logarithmic constant is trusted as an input; it is reconstructed by this rule.

4.4.3 Sine and cosine

Take the Maclaurin polynomial of degree N , with zero coefficients of the wrong parity. On an argument interval A , let $r = \max_{a \in A} |a|$. Since every derivative of sine or cosine has absolute value at most one,

$$|f(a) - T_N(a)| \leq \frac{r^{N+1}}{(N+1)!} \quad (a \in A). \quad (12)$$

The polynomial is evaluated by exact interval arithmetic and enlarged by that rational error. Intersecting with the known range $[-1, 1]$ is sound and often useful.

The last intersection explains a useful automatic proof improvement. To prove $1 - \cos x \geq 0$, a checker does not need the Taylor polynomial to resolve the exact zero at $x = 0$: the universal upper bound $\cos x \leq 1$ already preserves it. The jet selector can exploit this fact when a higher derivative has that form.

4.5 Coverage certificates rather than sampled grids

A certificate tree starts from the problem's original interval. A leaf records a series order and an advertised rational enclosure. The checker recomputes an enclosure of the original expression on that leaf, and accepts only if the advertised enclosure contains the recomputed one and has nonnegative lower endpoint.

An internal node records a rational split point s strictly between the current endpoints. Its children are interpreted as $[\ell, s]$ and $[s, u]$. The checker derives these child domains itself. It does not trust independently supplied child boxes, so there is no opportunity to omit a small interval around a troublesome point.

Theorem 4.1 (Interval-tree soundness). *Assume the elementary atom bounds are valid and the interval operations preserve enclosure. An accepted coverage tree proves $e(x) \geq 0$ for every x in the original interval, with all required analytic-domain conditions satisfied there.*

Proof. Structural induction on the expression gives enclosure on each checked leaf. Its nonnegative lower endpoint proves the local inequality. Induction on the tree combines the two child results, using the exact interval-cover identity at every split. The original-expression domain traversal supplies the side conditions on all leaves and hence on their union. $\square$

The default proposer tries orders 8, 16, 32, then bisects the interval if needed. It has a depth and node budget. A finite precision ceiling is not a completeness guarantee. A certificate produced by another search policy can still be accepted, provided its recorded orders and tree fit the checker's explicit limits.

The workload includes twelve positive-margin expressions of the form $x - x + \varepsilon$. The input AST deliberately retains both occurrences of x , so natural interval arithmetic gives $[\ell - u + \varepsilon, u - \ell + \varepsilon]$. Subdivision is needed until the leaf width is small enough. These are transparent coverage tests, not hard mathematical inequalities; a symbolic simplifier would close them immediately.

4.6 Vanishing jets: remove the uncertain value at the zero

A jet certificate records an anchor a , an order $m \geq 1$, exact vanishing conditions, and an interval-tree proof of a sign condition on the m -th derivative.

Theorem 4.2 (Zero-aware nonnegativity). *Let f be C^m on a neighborhood of a closed interval B , let $a \in B$, and suppose*

$$f(a) = f'(a) = \dots = f^{(m-1)}(a) = 0, \quad f^{(m)}(x) \geq 0 \quad (x \in B). \quad (13)$$

If m is even, then $f(x) \geq 0$ throughout B . If m is odd, the same conclusion holds on $B \cap [a, \infty)$.

Proof. At $x = a$, the conclusion is an exact vanishing condition. Otherwise Taylor's theorem gives a point ξ strictly between a and x such that

$$f(x) = \frac{f^{(m)}(\xi)}{m!}(x - a)^m.$$

The derivative factor is nonnegative and the factorial is positive. The power is nonnegative for even m , or for $x \geq a$ with arbitrary m . Multiplying gives the claim. $\square$

The prototype's odd-order rule requires the whole input interval to lie to the right of the anchor. A left-sided odd-order certificate could instead use $f^{(m)} \leq 0$; that symmetric constructor is not implemented. Rejecting an odd right-oriented certificate on $[-1, 1]$ for $f(x) = x^3$ is one of the unit tests.

The exact-zero condition is stronger than an interval containing zero. The anchor evaluator accepts rational arithmetic together with the exact identities

$$\exp(0) = 1, \quad \log(1) = 0, \quad \sin(0) = 0, \quad \cos(0) = 1.$$

If an anchor expression cannot be evaluated by those rules, the attempt is inconclusive. A high-precision approximation close to zero is never converted to an exact jet coefficient.

4.7 Four worked certificates

Exponential tangent. For $f(x) = e^x - 1 - x$, the anchor values $f(0) = f'(0) = 0$ are exact and $f''(x) = e^x$. On $[-1, 1]$, a rational exponential lower enclosure is positive. The order-two certificate proves

$$e^x \geq 1 + x.$$

The point $x = 0$ is handled by equality, not by a numerical tolerance.

Logarithmic tangent. For $f(x) = x - \log(1 + x)$, the same two anchor coefficients vanish and

$$f''(x) = \frac{1}{(1 + x)^2} > 0$$

on $[-1/2, 1]$. The original domain check proves $1 + x \geq 1/2$; the reciprocal interval therefore excludes a pole. This gives

$$\log(1 + x) \leq x.$$

Cosine lower approximation. Set $f(x) = \cos x - 1 + x^2/2$. Its first four jet coefficients at zero vanish, and $f^{(4)}(x) = \cos x$. A positive cosine enclosure on $[-1, 1]$ gives the recorded order-four certificate for

$$\cos x \geq 1 - \frac{x^2}{2}.$$

A shorter proof also exists: $f'' = 1 - \cos x \geq 0$, so order two suffices. The automatic selector finds that lower-order certificate by exploiting the universal trigonometric range bound.

Sine lower approximation. Let $f(x) = \sin x - x + x^3/6$. The first five jet coefficients vanish at zero and $f^{(5)}(x) = \cos x$. On $[0, 1]$, the right-sided order-five rule proves

$$\sin x \geq x - \frac{x^3}{6}.$$

Again a lower-order proof is possible: $f''' = 1 - \cos x \geq 0$. The automatic selector finds an order-three certificate on the right-sided interval.

The manual-order and automatic-order certificates concern the same four mathematical goals and are not counted as additional independent examples. Their difference is useful nonetheless: it demonstrates that a certificate's jet order need not equal the function's exact order of vanishing. Any lower order satisfying the theorem's hypotheses is sufficient.

4.8 Automatic proposal without guessing exact zeros

The additional `auto_prove` entry point first attempts a shallow interval tree. If that fails, it tries the anchor set consisting of zero and the two rational interval endpoints, removing duplicates and anchors outside the domain. At each anchor it differentiates repeatedly, checking exact vanishing after each step, up to a configured maximum order.

If a required coefficient is nonzero or unsupported by the exact evaluator, larger orders at that anchor cannot justify the same vanishing certificate and are not explored. An odd order with an interval extending left of the anchor is skipped. Every admissible order then gets a bounded derivative-sign proof attempt.

```
try_exact_jets(expression, interval):
    try bounded ordinary interval proof
    for anchor in {0, left endpoint, right endpoint}:
        current := expression
        for order from 1 to 6:
            if exact_value(current, anchor) is not exactly zero:
                stop this anchor
            current := symbolic_derivative(current)
            if orientation is admissible:
                try a bounded interval proof current >= 0
                on success return the jet certificate
    return UNKNOWN
```

This selector was executed on the four worked goals and independently checked. It found orders 2, 2, 2, 3, respectively. The count of attempted proof profiles was 2, 2, 2, 4, including the failed initial interval attempt. These are recorded separately in `results/jet-discovery.json`.

More ambitious anchor proposal can use approximate minimizers or roots of a polynomial surrogate, but approximate zeros must then be turned into exact anchor identities or into isolating-interval arguments. A candidate numerical minimum with value near zero is useful guidance and nothing more. Algebraic anchors would require an exact algebraic-number representation and a bridge to the existing polynomial root machinery.

4.9 The exact meaning of checker independence

The search and checker are separate modules. The search uses simplified symbolic derivatives; the checker derives the expression independently with unsimplified differentiation rules. Search-side Taylor sums use powers and factorials; checker-side coefficients are recomputed by recurrences. Both use Python's exact rational arithmetic and the same mathematical atom-bound specifications.

Consequently this is meaningful producer/checker separation, but not a formal correctness proof. A mistaken mathematical bound shared by both implementations could escape these tests. Nor is the checker's unsimplified derivative always as precise as the searcher's simplified interval expression. It may therefore reject a sound candidate whose advertised enclosure is too narrow for its own evaluation. That is a false negative, not permission to widen the accepted semantics. A production reflected derivative DAG with proved simplification would remove much of this mismatch.

The optional numerical crosscheck evaluates 320 rational point arguments with 80 decimal digits using `mpmath`. It is explicitly outside acceptance, uses a small tolerance only for comparison at exact numerical endpoints, and is reported as a sanity check rather than mathematical proof. The certificates themselves use no `mpmath`, floating-point transcendental routine, or external numeric library.

4.10 A limited completeness theorem

Theorem 4.3 (Robust strict positivity, idealized search). *Consider an admitted elementary AST on a compact rational interval, with every logarithm argument bounded away from zero and every divisor bounded away from zero. If its denotation is strictly positive throughout the interval, an unbounded fair search combining arbitrarily fine subdivision and arbitrarily high series orders eventually finds a finite interval certificate.*

Proof. Continuity and compactness give a positive minimum δ . The domain margins persist in sufficiently small neighborhoods. At each point, the elementary rational enclosures converge as the order increases, and the natural interval extension converges to the point value as the interval shrinks. By structural induction on the finite AST, a sufficiently small rational neighborhood and sufficiently high order give a lower enclosure above zero. Compactness supplies a finite subcover. A sufficiently fine subdivision refines these neighborhoods, and sufficiently large per-leaf orders establish positivity on the resulting finite partition. Fair exploration eventually reaches such choices. $\square$

This theorem does not apply to arbitrary nonnegative functions with zeros. The jet rule covers a useful additional class but is not a general nonnegativity decision procedure. It also does not apply to the literal fixed order set $\{8, 16, 32\}$, fixed depth caps, or fixed jet order limit in the supplied executable. Distinguishing a limiting algorithm from its budgeted implementation is necessary for an honest capability statement.

4.11 A stronger production lane: polynomial Taylor models

Natural interval arithmetic loses dependencies between repeated subexpressions. The next step should reuse Forge’s existing polynomial engines rather than implement another polynomial positivity checker. On a rational box, represent an expression by

$$e(x) = P(x) + \rho(x), \quad \rho(x) \in I,$$

where P is a rational polynomial and I a rational interval. Addition combines the polynomials and remainder intervals. Multiplication keeps the polynomial product, truncates it only with a separately bounded tail, and bounds the terms involving remainders. Elementary composition expands around a chosen rational center and attaches a proved tail bound.

A goal $e \geq 0$ becomes the polynomial obligation $P + \inf I \geq 0$, which the existing cone, Bernstein, or Sturm workers may settle. Near a zero, a jet or exact factorization should be applied before discarding correlation into an interval remainder. Otherwise a sophisticated Taylor model can reproduce the same zero obstruction as a simple interval evaluator.

This Taylor-model implementation is not included. The current atom enclosures and coverage checker establish a smaller integration target first. Mathlib’s real-variable Taylor library already exposes Lagrange and integral remainder results, including `taylor_mean_remainder_lagrange_iteratedDeriv`; those are concrete theorem-bridge targets, not a claim that their precise signatures were compiled against the repository’s pinned Mathlib revision [17].

4.12 Library choices and range reduction

For search, `python-flint`'s `arb` type is a useful optional interval proposal engine: its documented representation uses a floating-point midpoint and an outward radius, and its precision is configurable [18]. A Forge adapter can use such balls to select subdivisions, expansion orders, and centers. It must still export rational bounds or an explicit analytic certificate accepted by the Lean-side checker. An `Arb` interval is not by itself a Lean proof.

The reference formulas work best for moderate arguments. Large exponential arguments should be reduced by identities such as $\exp(x) = \exp(x/2^k)^{2^k}$, with repeated squaring checked explicitly. A positive logarithm argument can be scaled into a compact range and accompanied by a separately certified $\log 2$ term. Trigonometric reduction requires certified bounds involving π and proof that the chosen reduction index is valid; a floating-point quadrant computation alone is not enough. None of these range-reduction optimizations is present in the prototype, so large arguments can legitimately produce `UNKNOWN` under its fixed caps.

5 Integration into Forge, Leant, and Djex

5.1 Keep the existing controller; add three typed adapters

Forge's reviewed runtime contract already distinguishes a candidate, a conditional candidate with premises, and a no-result outcome. It also carries a scope key and resource limits. The new workers should attach to that design rather than define a competing global proof state [6].

The following proposed module layout is deliberately local:

```
Forge/Polyhedral/Reify.lean
Forge/Polyhedral/Rows.lean
Forge/Polyhedral/ProjectionCheck.lean
Forge/Polyhedral/Lift.lean
Forge/Polyhedral/Replay.lean

Forge/Noncomm/Reify.lean
Forge/Noncomm/WordPoly.lean
Forge/Noncomm/IdentityCheck.lean
Forge/Noncomm/Replay.lean

Forge/Analytic/Expr.lean
Forge/Analytic/Domain.lean
Forge/Analytic/Interval.lean
Forge/Analytic/ElementaryBounds.lean
Forge/Analytic/Derivative.lean
Forge/Analytic/Jet.lean
Forge/Analytic/Replay.lean
```

These are proposed paths, not files that already exist in the source repository or in the supplied Lean specimens. The corresponding Python modules are real and are listed in Appendix A.

A worker should be triggered by a semantic classifier, not only by a visible token. An existential real-valued tuple with affine constraints is a polyhedral job. An equality over a recognized associative algebra with useful local polynomial relations is a noncommutative job. A real inequality containing admitted elementary atoms and a proved bounded domain is an analytic job. Unsupported pieces remain separate obligations or cause a refusal of the fragment.

5.2 The polyhedral source bridge

The source bridge must record the correspondence between each parameter/output coordinate and its Lean local expression, as well as a proof of every affine normalization. It should first target $\mathbb{R}$, with rationals embedded by their exact casts. Equalities are translated to two rows only with the elementary equivalence theorem. A strict inequality must never be normalized to a non-strict row merely to fit an existing data type.

The principal theorem needed for replay has the shape

```
checkProjection problem certificate = true ->
  forall x, domain problem x ->
    relation problem x (extractWitness problem certificate x)
```

The exact Lean representation can use vectors or finite functions. The certificate's output order and the witness extractor's dependency order must be one and the same checked schedule.

The resulting witness is an explicit mathematical term, not an appeal to an unspecified choice operator. That does not imply that arbitrary real-number comparison and arithmetic become executable machine operations. A Lean term over $\mathbb{R}$ may be noncomputable in the code-generation sense. A corresponding $\mathbb{Q}$ instance of the same ordered-field construction is a natural executable specialization, but the source type must not silently change from reals to rationals.

Negative replay needs a stronger source bridge than positive replay. Suppose the reifier treats $\sin t$ and $\cos t$ as independent real parameters. A proof valid for all independent values is safe to instantiate back into the source. A counterexample using impossible independent values is not a counterexample to the source theorem. Similarly, dropping some local hypotheses is safe for a positive proof from the remaining hypotheses, but unsafe for a refutation that ignores them.

Therefore the polyhedral counterexample constructor should initially be exposed only when all parameter coordinates are actual independent real binders and the reified domain contains the full relevant premise. Otherwise its negative receipt is a diagnostic about the abstraction, not a source-level disproof. The Python problem format has exact independent coordinates; this qualification arises when connecting it to Lean expressions.

5.3 The noncommutative source bridge

The reifier starts by selecting one multiplication carrier and one interpretation environment for its atoms. Rational scalar casts, numeral powers, multiplication order, and subtraction are normalized into word polynomials with accompanying interpretation lemmas. A hypothesis is registered as a relation only when its normalized polynomial is proved to evaluate to zero.

The checker theorem should quantify over the interpretation rather than hard-code matrices or operators:

```
checkIdentity relations target certificate = true ->
  (forall i, eval relations[i] rho = 0) ->
    eval target rho = 0
```

This makes the same certificate usable in different algebras while keeping its original source hypotheses explicit. A new relation discovered by completion is not inserted into the local context until its original-relation receipt has been replayed.

The first implementation can emit source-level contextual equalities and rely on Mathlib normalization. That is a useful bring-up path, particularly for the short critical-pair example. It should not be mistaken for a scalable reflected checker: a long sum of contextual hypotheses can create enormous elaboration and normalization work. The sparse coefficient checker and its general interpretation theorem are the appropriate second step.

5.4 The analytic source bridge

Analytic replay needs four proved layers, each with a narrower responsibility than a general tactic:

1. An AST denotation and a theorem connecting it to the original Lean expression.
2. Exact interval-operation and elementary-atom enclosure theorems, including denominator and logarithm domains.
3. A symbolic derivative theorem and an exact anchor-evaluation theorem.
4. Coverage-tree and jet soundness theorems that compose those local facts.

The proof of a jet certificate cannot consist only of checking a list of rational values. It must establish that the differentiated AST denotes the derivative of the original function, and that the original function has the required smoothness on the interval. This is why the analytic port is a larger formalization task than its short Python search routine suggests.

The real-variable Taylor results in Mathlib are an existing bridge for the last layer. Their documented formulation uses real intervals and iterated derivatives; generalization to higher-dimensional domains is not something this report assumes already solved by the same API [17]. For the initial univariate worker, prove reusable corollaries with the exact interval orientation and vanishing hypotheses needed by Theorem 4.2, so later certificates do not reconstruct the analytic argument from scratch.

5.5 Concrete cooperation: a witness plus an analytic guarantee

Consider strengthening the four-corner problem to

$$\exists z, \quad \max(0, x + y - 1) \leq z \leq \min(x, y) \quad \wedge \quad e^z \geq 1 + z,$$

under $0 \leq x, y \leq 1$. A useful plan has three proof steps. The polyhedral worker constructs z for the linear part. Its output bounds imply $0 \leq z \leq 1$. The analytic worker supplies a theorem for $e^t - 1 - t \geq 0$ on $[0, 1]$, which is instantiated at the constructed witness. The existing local engine then assembles the conjunction.

The analytic conjunct is not simply omitted while solving the existential. The controller must retain it as an obligation and use a proved implication from the linear postcondition to that obligation. Abstractly, the composition rule is

$$[D(x) \Rightarrow L(x, W(x))] \wedge [D(x) \wedge L(x, y) \Rightarrow A(x, y)] \implies D(x) \Rightarrow \exists y, L(x, y) \wedge A(x, y).$$

This plan is mathematically justified here but has not been run through a Lean obligation broker. It is a concrete integration test to add after the individual replay theorems exist.

5.6 A longer-term operator application with an explicit bound

The algebraic and analytic workers can also cooperate on operator approximations, but an additional norm/series bridge is needed. A precise target illustrates both the opportunity and the missing work.

Let A, B lie in a unital real or complex Banach algebra, let $h \geq 0$, and put $s = h(\|A\| + \|B\|)$. Comparing terms of total degree at most two gives the exact noncommutative identity

$$[e^{hA}e^{hB}]_{\leq 2} - [e^{h(A+B)}]_{\leq 2} = \frac{h^2}{2}(AB - BA),$$

where the brackets mean truncation of the power series, not an equality of full exponentials. Bounding the tails by submultiplicativity yields

$$\begin{aligned} & \left\| e^{hA} e^{hB} - e^{h(A+B)} - \frac{h^2}{2}(AB - BA) \right\| \\ & \leq 2 \left(e^s - 1 - s - \frac{s^2}{2} \right) \leq \frac{e^s s^3}{3}. \end{aligned} \quad (14)$$

Indeed, the absolute scalar sum of the product-series terms of total degree at least three is bounded by the tail of $e^{h\|A\|+h\|B\|}$; the single exponential has the same upper tail bound because $\|A + B\| \leq \|A\| + \|B\|$. The scalar Taylor remainder is at most $e^s s^3/6$, giving the last inequality after adding the two tails.

The finite-degree cancellation is a word-polynomial job; the scalar tail bound is an analytic job. What is not yet supplied is the Lean theorem transporting scalar majorants through normed-algebra power series. Thus (14) is a concrete future composition target, not a tested capability of this prototype. It provides a more useful destination than an unspecified promise to “support physics”.

5.7 How Leant and Djex benefit without changing their type theories

Leant already reconstructs synthesized candidates in Lean, while Djex retains source-owned type information and carefully scopes the positive-only search families added beyond its complete structural core [7, 8]. Forge has already absorbed many of their general verification-boundary lessons. The relevant new transfer is more specific: these workers can become *contract-solving providers* for scalar or algebraic subproblems inside a synthesized candidate.

For example, a typed sketch may require a real-valued function w satisfying a mixed affine relation. Once the input/output binder telescope is fixed, the polyhedral worker can synthesize the function body and its postcondition together. This avoids enumerating many affine functions when the correct body needs max. The provider returns a term graph associated with that exact source selection, not an untyped expression string.

For a candidate implementation over an algebra, the noncommutative worker can discharge an equational contract using the source’s exact local relations. For a scalar elementary implementation, the analytic worker can discharge a bounded inequality contract. These are behavioral proof obligations; none licenses a typeclass dictionary to be erased or a universe selection to be changed.

The scope is first-order within an already selected source context. This report does not claim to solve Leant’s remaining Church-indexing composition or simultaneous universe-selection tasks. Nor can a counterexample to one fixed arithmetic implementation be turned into a proof that no term of the entire higher-order type exists. Candidate validation and global inhabitation remain distinct questions, as they are in the reviewed projects.

6 Executed evidence and its limits

6.1 Environment and reproducibility

The experiments were executed with Python 3.13.5 on Linux x86-64, using seed 20260915. Search, certificate checking, witness extraction, the exact vertex oracle, and the unit suite use the standard library. A separate optional script uses `mpmath` for numerical sanity checks. Exact interpreter and platform strings are recorded in `results/summary.json`.

No Lean or Lake executable was available in this execution environment. The supplied Lean examples have not been compiled, and no new Lean-kernel acceptance result is claimed. The reviewed repository’s own historical elaboration receipts are separate evidence and are not reused as validation of this study’s new code.

The experiment runner writes to `reproduced-results/` by default. The delivered evidence is under `results/`; reproducing an experiment therefore does not silently replace the recorded run. All certificates are stored together with their original externally checked problems. The generator and checker sources are included, rather than only a summary table.

6.2 Keep the units and outcomes separate

Workload	Cases	Positive receipts	Refutations	Unknown
Polyhedral specifications	110	79	30	1
Noncommutative identities	85	82	0	3
Analytic inequalities	44	37	0	7

Table 2: Recorded outcomes. Analytic positives comprise 33 interval and four jet certificates. A positive receipt here means successful exact Python replay, not a Lean proof.

Every expected outcome was obtained. That is not a theorem-proving success rate on an independent benchmark: the workloads were generated to exercise known mechanisms and controls, and several cases are parameterized variants of the same mathematical family. The rows should not be pooled into a single “goals solved” headline. In particular, the negative and budget-limited cases were designed to be reported as negative or unknown.

6.3 Polyhedral workload

The 79 positive cases consist of forty scaled four-corner specifications, thirty strict interval-selection specifications, one touching-extrema strictness case, three one-sided or unconstrained-output cases, and five empty-domain cases. Half of the corner specifications add a coupled second output. The thirty negative cases have an exact rational parameter counterexample; the remaining case deliberately sets the pair budget to zero.

The witness extractor was evaluated at 490 rational parameter tuples across the corner and strict-selection families. Every value agreed with the reverse-lifting evaluator and satisfied all target rows under an independent row evaluation. This is an execution check of the generated function graphs, not the reason the universal specifications are accepted: the projection and implication receipts carry that reason.

A separate exact feasibility workload contains 160 bounded two-variable mixed systems. It is compared with a different algorithm: introduce a common strict slack $\delta \in [0, 1]$, replace each strict row by a non-strict row strengthened by δ , and enumerate vertices of the resulting bounded polyhedron using rational Gaussian elimination. A strict feasible point exists exactly when the maximum possible slack is positive; on a nonempty compact polyhedron a linear maximum is attained at a vertex. For systems without strict rows, ordinary feasible-vertex existence suffices.

The oracle reports 89 feasible and 71 infeasible systems, agreeing with Fourier–Motzkin on all 160. The original variables are explicitly boxed, so the compactness premise of the vertex oracle is not an unstated assumption. These 160 systems are not the 110 quantified specifications and are not counted as additional synthesized functions.

6.4 Noncommutative workload

The 85 cases comprise the completion-disabled/enabled pair, 36 Weyl instances, forty quantum-plane words, five Clifford instances, and two additional unknown controls. The positive count is 82. The controls ensure that commutativity is not silently assumed and that the false consequence $DX - XD = 0$ is not accepted under $DX - XD = 1$.

The completion-disabled case is also UNKNOWN. It is not labeled a refutation: its target is true and the completion-enabled run proves it. The two runs therefore isolate an actual capability difference between the prototype’s search profiles without claiming anything about a different tactic implementation.

The 288 polynomial-action checks for the Weyl family use a representation different from formal word expansion. They are useful differential evidence, but only a finite set of input monomials is evaluated. The accepted certificates themselves are identities in the free algebra, which gives the stronger conclusion for each concrete (m, n) instance.

6.5 Analytic workload and automatic rediscovery

The 44 cases comprise twenty exponential positive-margin inequalities, twelve explicit dependency/subdivision tests, one nested exponential/logarithm example, four plain-interval attempts on zero-containing goals, the corresponding four jet proofs, and three false/domain controls. The ordinary interval profile returns UNKNOWN on each of the four zero-containing tests under the recorded depth cap, while the selected jet profiles are accepted.

The controls include a false exponential inequality, a logarithm crossing its admitted domain, and a division whose denominator crosses zero. No analytic refutation is emitted for them. The first is a false goal; the other two are fragment/domain failures, and the implementation does not conflate those reasons into a universal false verdict.

The automatic selector was then run on the same four successful jet goals. It found orders 2, 2, 2, 3, with certificates checked independently. Those results are stored separately in `results/jet-discovery.json`. They are an overlapping search-profile experiment, not four more independent mathematical cases.

The 320 optional 80-digit point checks cover exponential, logarithm, sine, and cosine at rational arguments. Their output file is explicitly labeled NUMERICAL_SANITY_ONLY. They are not invoked by the certificate checker.

6.6 Mutation and structural controls

Lane	Rejected	Representative mutations
Polyhedral	180	Wrong eliminated variable, missing pair coverage, changed output coefficient, lost strictness, invalid counterexample-row index
Noncommutative	81	Perturbed provenance coefficient, changed target, non-canonical rational or floating-point coefficient
Analytic	81	Invalid series order, escaped anchor, invalid jet order, changed target, false advertised bound, coverage gap

Table 3: Generated mutation attempts, all rejected. These are test counts, not a claim of parser hardening or a measure of independent bug classes.

The unit suite contains twelve named tests, including all-recorded-case replay as subtests. The targeted checks cover a zero weight on a strict row, missing projection coverage, left/right noncommutative context order, odd-jet orientation, original-expression domain retention, and invalid interval coverage. All twelve tests pass.

The mutation counts should not be mistaken for adversarial security certification. Many are parameterized repeats of the same fault shape. The shared rational parser imposes some canonicity and bit-length checks, but the tools are not hardened against deliberately enormous or deeply adversarial external inputs. In particular, byte limits before JSON parsing, global

memory limits, and comprehensive exception isolation are production requirements, not properties demonstrated by these runs.

6.7 Replay with search unavailable

The replay command installs an import hook that rejects imports of the three search modules and the witness-proposal module. It then loads the independent checkers and the stored problems/certificates. Running it with `python -S` also disables site packages.

```
cd prototype
python -S verify.py
python -S verify.py ../results/jet-discovery.json
python -S -m unittest discover -s tests -v
python -S run_experiments.py --out reproduced-results
python -S discover_jets.py --out reproduced-jets.json
```

The default recorded replay accepts 109 polyhedral receipts (79 positive and thirty negative), 82 noncommutative receipts, and 37 analytic receipts. It rejects the corresponding unknown objects as noncertificates. This verifies that the checkers do not secretly call the producer entry points during these replays.

Producer/checker independence is nevertheless bounded. All programs run on the same Python runtime and exact rational implementation. The analytic atom formulas are a shared mathematical specification. The tests do not prove the checkers correct. The intended next evidence level is a Lean theorem relating each reflected Boolean check to its semantic proposition.

6.8 Timing and size data

Per-case search and replay wall times, as well as serialized certificate byte counts, are available in `results/measurements.csv`. These are single local runs of deliberately small research fixtures. They do not measure Lean elaboration, proof-term construction, kernel checking, source reification, process startup, or IDE cancellation behavior. No speedup relative to `grind`, `Aesop`, `linarith`, `noncomm_ring`, or another tactic is inferred.

The serialized certificate totals by lane are 65,364 bytes for the polyhedral workload, 96,989 bytes for noncommutative identities, and 17,592 bytes for analytic cases, using the runner's compact JSON serialization for each object. The pretty-printed artifact file also includes the original problems and therefore has a different size. These byte totals describe exactly what the runner serialized; they are not estimates of Lean proof sizes.

6.9 What the evidence establishes

The supplied data establish that these concrete implementations find and independently replay the reported certificates, that the specified control cases receive the intended outcomes, and that several distinct computations agree on overlapping finite tests. The paper supplies mathematical proofs for the certificate rules and the exact fragment-level claims.

The evidence does not establish a working Forge tactic, a verified source reifier, a formally verified Python or Lean checker, broad benchmark superiority, or robustness at production problem sizes. Those are separate engineering and evaluation tasks, with explicit release gates in the next section. Keeping them separate allows the actual implemented advances to remain useful without overstating them.

7 An implementation sequence with falsifiable release gates

7.1 Gate 1: promote exact polyhedral receipts to Lean theorems

The first deliverable should not be a broad forge command with a long list of unsupported options. It should be a small command or tactic that replays one explicit polyhedral certificate against a source goal. Prove the row-combination theorem, strictness propagation, complete one-variable lifting, and reverse-schedule witness correctness. Connect the source reifier only after those theorems exist.

The acceptance examples must include the non-affine four-corner specification, at least one simultaneous-output specification, open intervals, touching extrema with nonactive strict bounds, an empty domain, and a genuine source-level counterexample. The negative gate must explicitly exclude counterexamples to lossy atom abstractions. A solution that proves only the closed non-strict one-output fragment does not close this gate.

Once replay works, connect the current Python producer or an in-process equivalent. The proof-producing boundary should accept the same problem and certificate regardless of which search backend proposed it. Before replacing the deterministic pivot order, preserve its results as a regression profile.

7.2 Gate 2: noncommutative certificates before completion in Lean

Start with a hand-supplied certificate in the form (5). Prove the sparse polynomial evaluator correct and show that every contextual original-relation term vanishes. A successful kernel replay should not depend on trusting a claim that the returned rules are a Gröbner basis.

The acceptance gate includes (6), Weyl and Clifford specimens, a noncommuting left/right context mutation, a nonzero rational coefficient mutation, and a changed target. It must also demonstrate that a nonzero remainder returns no proof of disequality. Then attach the Python completion worker; an optional GBNP adapter can be added without changing the checker theorem.

Large flattened receipts should be measured before implementing a DAG format. If proof construction rather than search becomes the bottleneck, preserve sharing at the certificate layer and prove the DAG interpreter once. The optimization must not replace original relation indices with unaudited derived-rule names.

7.3 Gate 3: analytic atoms, then exact jets

Prove one elementary atom family at a time. A useful order is rational interval operations, exponential and logarithm bounds, trigonometric bounds, expression denotation, derivative correctness, then the jet constructor. The domain checker should be in place before algebraic simplification is admitted at the source boundary.

The gate is not closed by a source proof that calls a preexisting theorem for $e^x \geq 1 + x$ while bypassing the new certificate. Such a proof is a valid theorem but does not validate the proposed checker. The accepted result must replay the same series orders, interval tree, and vanishing conditions that the worker emitted. Conversely, the baseline comparison should be allowed to use ordinary library lemmas, so it does not artificially hide existing automation.

At least one zero-containing inequality must be proved by the automatic selector, not only by a manually supplied jet order. A sign-changing odd-power control, a false inequality, a missing analytic domain, a coverage gap, and an inexact anchor-zero claim must be rejected. The initial gate is univariate; multivariate Taylor models are a separate release criterion.

7.4 Gate 4: composition through the obligation broker

The witness-plus-exponential example in Section 5 should become an end-to-end test: source reification, polyhedral synthesis, domain propagation, analytic replay, and final conjunction assembly. Every accepted result must close the original source goal, not a weakened linear projection of it.

For Leant/Djex integration, select a source-owned scalar contract and retain its binder telescope through witness extraction and term reconstruction. A generated body that checks only after changing the selected type, erasing a dictionary, or introducing an undeclared variable is a failure of the adapter, even if the body looks mathematically plausible.

This gate should include cancellation and worker failure. A timed-out producer may return `UNKNOWN`, but it must not leave a partially accepted lemma in the context. Existing Forge scope and admission machinery should enforce that behavior; there is no need to invent a second trust protocol for the three new workers.

7.5 Evaluation that would justify a stronger claim

The next corpus should separate four questions: how often the reifier accepts the intended fragment, how often the producer finds a certificate within its budget, how often reconstruction succeeds, and how much kernel work the resulting proof requires. A single solved/unsolved bit hides the most informative failures.

For polyhedral synthesis, hold out coefficient patterns and dependency graphs, not merely new scalar multiples of the corner example. Include exact comparisons with an independent polyhedral backend and cases that force several nested extrema. For noncommutative search, vary presentation, generator order, overlap depth, and coefficient growth; report completion growth and proof size separately. For analytic search, separate strictly positive goals, exact-zero goals, domain failures, and large-argument range-reduction cases.

A fair Lean comparison should record a local baseline portfolio, each new worker alone, the full cooperative configuration, and relevant ablations. It should include source reification, search, proof construction, and kernel checking in separate timing fields. Any timeout policy, supplied lemma set, and hint configuration should be identical where the comparison intends to isolate a mechanism.

None of these corpus-level claims is established by the present synthetic workloads. What the present study supplies is a concrete implementation and certificate basis on which such evaluation can be built.

7.6 Recommended scope decision

The most valuable immediate addition is constructive polyhedral projection: it replaces an incomplete witness grammar with a complete algorithm for a clearly delimited but useful fragment. Noncommutative completion comes next because its proof object is exceptionally simple relative to the power of its search. The analytic worker should follow with a narrow, fully proved atom library and exact jets before a larger catalog of functions.

Together they extend Forge along three different axes without replacing its existing strengths. They construct witnesses its affine templates cannot express, discover algebraic consequences its initial rewrite set may not expose, and establish analytic inequalities outside its polynomial fragment while respecting exact zeros. The mathematical rules, executable prototypes, and recorded failures-to-prove all fit the same design principle already present in Forge: make a useful search result small enough that a much simpler checker can justify exactly what it means.

A Artifact guide and source provenance

A.1 What is in the archive

Path	Purpose
article/forge-extensions.tex	Main editable source for this report
article/sections/	Technical sections and artifact guide
article/forge-extensions.pdf	Rendered report
prototype/forge_delta/poly_search.py	Exact mixed Fourier–Motzkin search and lifting
prototype/forge_delta/poly_check.py	Independent projection and contradiction replay
prototype/forge_delta/poly_witness.py	Checked schedule to shared min–max expression graph
prototype/forge_delta/nc_search.py	Traced bounded noncommutative completion
prototype/forge_delta/nc_check.py	Direct original-relation identity checker
prototype/forge_delta/analytic_search.py	Elementary enclosures, subdivision, jets, automatic selector
prototype/forge_delta/analytic_check.py	Independent domain, derivative, enclosure, and coverage replay
prototype/run_experiments.py	Deterministic generators, controls, and measurements
prototype/verify.py	Replay with search imports prohibited
prototype/tests/	Unit tests and independent exact oracles
prototype/discover_jets.py	Overlapping automatic-jet search-profile experiment
prototype/numerical_crosscheck.py	Optional numerical sanity checks, outside acceptance
results/	Problems, certificates, measurements, mutation outcomes, and execution logs
lean/	Uncompiled source-level specimens and composition lemmas
docs/STATUS.md	Implementation and acceptance ledger
docs/sources.json	Revision pins, reviewed source paths, and external references

The archive contains newly authored material, not copies of the eighteen earlier submissions or their PDFs. No font files or third-party package source are redistributed. The referenced projects retain their own licensing terms.

A.2 Running the supplied code

From the extracted archive:

```
cd prototype
python -S -m unittest discover -s tests -v
python -S verify.py
python -S verify.py ../results/jet-discovery.json
python -S run_experiments.py --out reproduced-results
python -S discover_jets.py --out reproduced-jets.json

# Optional, outside proof acceptance:
python -m pip install mpmath
python numerical_crosscheck.py --out numerical-crosscheck.json
```

The reference environment used Python 3.13.5. The code uses ordinary modern Python features and exact `Fraction`; other versions should be tested rather than assumed bit-for-bit identical. No third-party package is required for search or replay.

To build the article with a reasonably complete TeX Live or MiKTeX installation:

```
cd article
pdflatex -interaction=nonstopmode -halt-on-error forge-extensions.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-extensions.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-extensions.tex
```

The font packages are standard TeX packages; there is no shell escape, bibliography tool, or font-file download step in the build.

A.3 Pinned repositories and reviewed surfaces

Forge

c98e47c5f804e92880fc1d0e378c1b95832b685c. Reviewed README, implementation status, provenance, consolidated witness and nonlinear sections, Leant/Djex transfer document, and runtime contract. The repository records a Lean v4.34.0 baseline and Mathlib revision 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa.

Leant

6bf05ad78c467989e68290f2d08bbbed40802d485. Reviewed README and its current synthesis scope/priorities. This is contextual evidence for the adapter design, not a new test of Leant’s historical acceptance ledger.

Djex

e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. Reviewed the canonical rank-N document’s opening rules and scope account. No Djex build or synthesis suite was run in this study.

The Mathlib and external-library documentation was viewed on 15 September 2026. Those are live documentation surfaces, not a claim that every displayed API was tested at Forge’s pinned Mathlib revision. Exact porting work should use the pin, compile the adapter, and record any necessary signature changes. The source manifest distinguishes repository snapshots from live documentation for that reason.

A.4 Reproduction and interpretation notes

The entry points named `prove` are untrusted proposers. Their status tags are meaningful only with the externally supplied problem and the appropriate checker; they do not construct a Lean proof object. Unknown outcomes are stored for audit and are intentionally rejected by certificate replay.

The diagnostic Lean-style expression emitted for the corner witness is a presentation artifact. Its existence is not evidence that a Lean elaborator accepted it. The same applies to the hand-written

files in `lean/`. Their headers state their uncompiled status, and they introduce no substitute sorry-based acceptance path.

The preserved results contain the final successful execution of the deterministic suite and the independent replay logs. During preparation, additional mutation cases and the automatic jet profile were added as separate validation increments. There was no failed full-suite run whose result was converted into a successful claim. Future reproductions should preserve failures alongside their command line and environment rather than overwrite them with a later success.

References

- [1] V. Reshetnikov, *Forge: implementation status*, snapshot c98e47c5f804, 2026. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/STATUS.md>.
- [2] V. Reshetnikov, *Forge consolidated article: provenance and merge ledger*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/B-provenance.tex>.
- [3] V. Reshetnikov, *Forge consolidated article: structural search and witness synthesis*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/05-structural.tex>.
- [4] V. Reshetnikov, *Forge consolidated article: nonlinear arithmetic*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/07-nonlinear.tex>.
- [5] V. Reshetnikov, *Ideas from Leant and Djex*, Forge documentation, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [6] V. Reshetnikov, *ForgeDesign runtime contract*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/lean/Forge/Design/Runtime.lean>.
- [7] V. Reshetnikov, *Leant: a Djex-based synthesis REPL for Lean 4*, README, snapshot 6bf05ad78c46, 2026. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [8] V. Reshetnikov, *Djex: rank-N and impredicative types*, snapshot e8778f4ebd63, 2026. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/docs/rank-n.md>.
- [9] Lean project, *The grind tactic*, Lean reference manual. Accessed 15 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [10] G. E. Liepins, *Fourier–Motzkin elimination for mixed systems*, Oak Ridge National Laboratory report, 1 August 1983. Bibliographic record and original report preserved by UNT Digital Library. <https://digital.library.unt.edu/ark:/67531/metadc1099248/>.
- [11] G. M. Bergman, *The Diamond Lemma for Ring Theory*, *Advances in Mathematics* **29** (1978), 178–218. Author’s bibliographic and correction page, including historical notes: <https://math.berkeley.edu/~gbergman/papers/updates/diamond.html>.
- [12] GBNP developers, *GBNP manual, Chapter 3: Functions*, especially trace variants `SGrobnerTrace`, `StrongNormalFormTraceDiff`, and `EvalTrace`. Accessed 15 September 2026. <https://gap-packages.github.io/gbnp/doc/chap3.html>.

- [13] Mathlib contributors, *Mathlib.Tactic.NoncommRing*, live API documentation. Accessed 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Tactic/NoncommRing.html.
- [14] Mathlib contributors, *Mathlib.LinearAlgebra.FreeAlgebra*, free-monoid basis and associated free-algebra structure. Accessed 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/FreeAlgebra.html.
- [15] A. Solov'yev and T. C. Hales, *Formal Verification of Nonlinear Inequalities with Taylor Interval Approximations*, 2013. <https://arxiv.org/abs/1301.1702>.
- [16] Interval project, *Interval Package for Rocq*, project documentation. Accessed 15 September 2026. <https://coqinterval.gitlabpages.inria.fr/>.
- [17] Mathlib contributors, *Mathlib.Analysis.Calculus.Taylor*, real-variable Taylor polynomials, Lagrange and integral remainders. Accessed 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Analysis/Calculus/Taylor.html.
- [18] python-flint developers, *arb—real numbers*, API documentation. Accessed 15 September 2026. <https://python-flint.readthedocs.io/en/latest/arb.html>.